

SISTEMAS DE TIEMPO REAL

Interrupciones

Interrupciones

- Son señales enviadas al microprocesador que causan la suspensión temporaria de lo que están haciendo para realizar otras tareas que requieren atención inmediata.
- Algunas interrupciones están destinadas a tareas comunes tales como captura de teclas o apertura de archivos. La consecuencia es que un programa es interrumpido y luego deberá reanudarse.
- Las interrupciones pueden ser:
 - Sincrónicas: son las generadas por la CPU al ejecutar instrucciones
 - Asincrónicas: son las generadas por otros dispositivos y no están alineadas al clock del sistema
- Intel:
 - excepciones = sincrónicas
 - interrupciones = asincrónicas

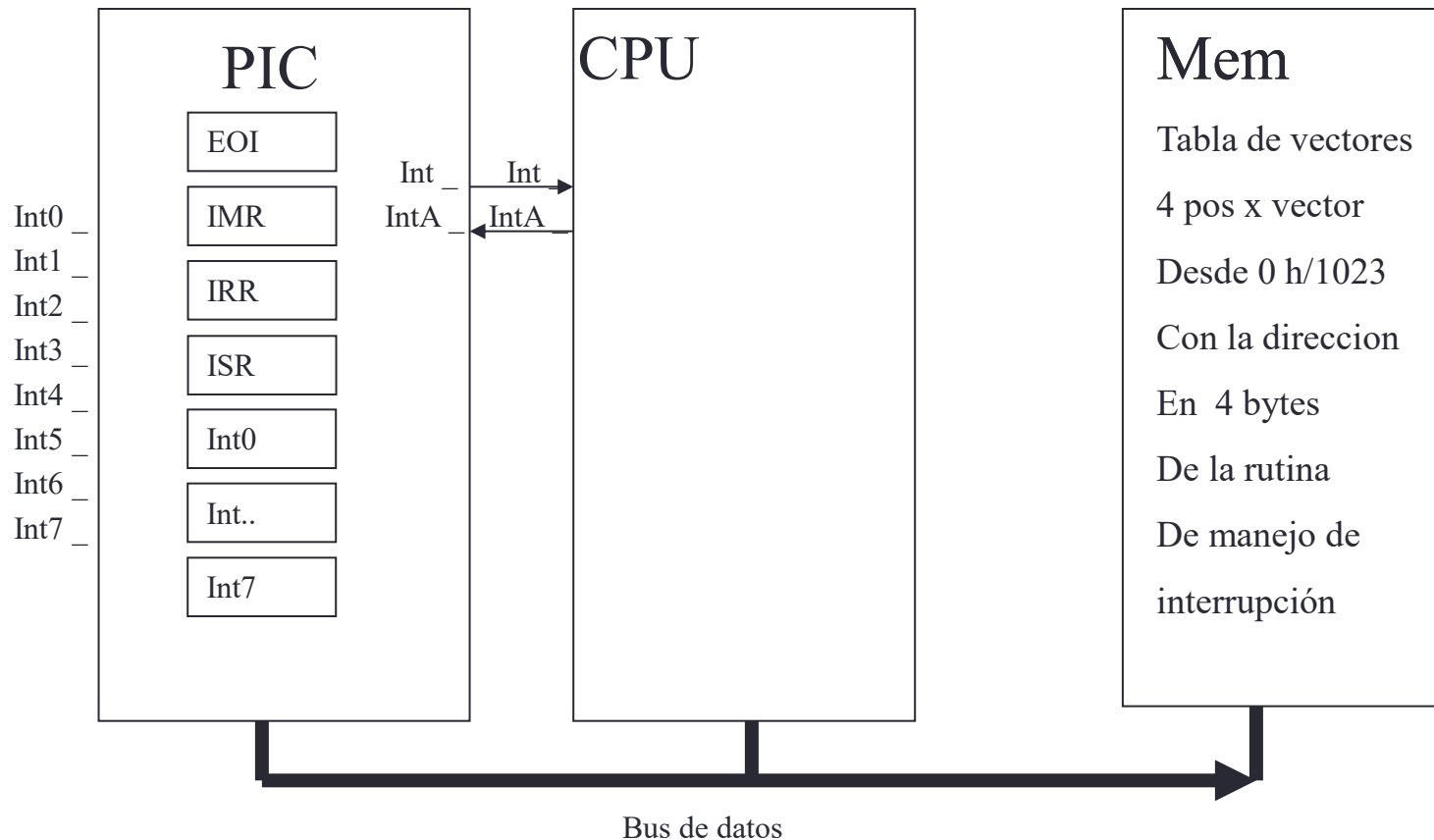
Interrupciones

- Intel proporciona una instrucción que deshabilita las interrupciones enmascarables
- Hay algunas que no puede deshabilitar (no enmascarables)
- Por lo general son errores de hardware
- El flag IF del registro EFLAGS determina el estado de las interrupciones
- Se garantiza que no se pierden interrupciones
- Estas son generadas luego que se habiliten nuevamente
- Cuando una interrupción es detectada por el procesador, se debe parar la ejecución de lo que se estaba haciendo y ejecutar el código correspondiente de atención de interrupciones
- Esto genera un *kernel control path*, que ejecuta en la estructura del proceso que estaba ejecutando
- Los registros de ejecución son salvados en el *Kernel Mode Stack del proceso*

Interrupciones de Hardware

- Son iniciadas por el hardware o por los circuitos de la placa del sistema, por una placa de expansión o a través de un puerto conectado a un dispositivo externo. Ejemplos de eventos que las inician son:
 - Pulsos del chip de timer de la computadora.
 - Una señal del módem.
 - Una señal del mouse.
- La atención de interrupciones debe satisfacer:
 - Las interrupciones deben ser atendidas de forma rápida, toda tarea que no sea necesaria se debe diferir para otro momento
 - Deben ser posible ejecutarlas en forma anidada
 - Deben ejecutar, todo lo que sea posible, con las interrupciones habilitadas
- En las arquitecturas antiguas estas señales de interrupción son manejadas por un chip controlador de interrupciones programable (PIC, programmable interrupt controller).

Interrupciones de Hardware antiguo



Interrupciones de Hardware antiguo

Información, que será guardada en la pila o stack

- El puntero a instrucción IP que apunta a la próxima instrucción a ser ejecutada. Este puntero indica un offset relativo a un valor almacenado en el procesador en el registro CS, segmento de código, y son estos dos valores (CS:IP) los que son almacenado en la pila, ocupando cuatro bytes. Debera ser almacenado para poder saber cual es la próxima instrucción a ejecutar al reanudar la ejecución del programa suspendido.
- El registro de flags del procesador, ya que en la atención de la interrupción se ejecutará un programa que modificará el estado de los mismos. Este registro ocupa dos bytes en la pila.
- Los registros del microprocesador que se vayan a usar durante la ejecución de la atención de la rutina, ocupando dos bytes en la pila cada registro.

Interrupciones de Hardware antiguo

- Las primeras dos informaciones son (CS:IP y flags) son almacenadas automáticamente en la pila cuando una interrupción ocurre.
- Los registros que se vayan a modificar deben ser guardados por el manejador de interrupción (interrupt handler), o rutina de servicio de interrupción (ISR, interrupt service routine). No son almacenados automáticamente pues algunos manejadores de interrupción alteran algún registro solamente. Al finalizar, la propia rutina deberá restablecer el valor de dichos registros y ejecutar un IRET (interrupt return) que restablece los otros registros almacenados en forma automática. Con el CS:IP restablecido, el programa interrumpido reanuda su funcionamiento nuevamente.

Interrupciones de Hardware antiguo

Chip controlador de interrupciones programable (PIC)

- El chip Intel 8259 controlador de interrupciones programable (PIC) es usado en las Pcs para manejar las interrupciones de hardware.
- Puede haber mas de un chip, en este caso el chip esclavo tiene su salida conectada a la IRQ2 del chip master.
- Las interrupciones tienen prioridades de acuerdo al numero. La IRQ0 se ejecutará antes, interrumpiéndola si ya esta en curso, que la IRQ1.

Funcionamiento del PIC

- El chip tiene 3 registros.
- El **registro de requerimientos** de interrupción cambia un bit a 1 en la posición de la interrupción requerida.
- El chip entonces consulta el **registro de interrupciones en servicio**, asegura el cumplimiento de las prioridades.
- Luego consulta al **registro de mascara de interrupciones** para ver si esta habilitada o enmascarada esa interrupción.
- Este ultimo registro es el único al que accede un programador, a través del port 21h para el chip master y a través del puerto A1h para el esclavo.
- Además esta el registro de comando de interrupciones, en el puerto 20h y A0h para ambos chips, que nos muestra que interrupción esta completada.

Acceso a los registros

Se usan las instrucciones

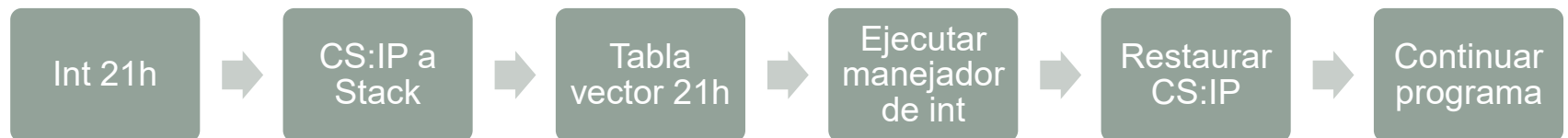
- ***INP*** y ***OUT*** en Basic
- ***port*** en Pascal
- ***outporb*** e ***inportb*** en C Borland
- ***inp*** y ***outb*** en C microsoft
- ***In*** y ***Out*** en Assembler.

Interrupciones de Software

- Son iniciadas por el software.
- Un programa llama una interrupción, por ejemplo para realizar la tarea de colocar un carácter en la pantalla.
- Como es el software el que decide cuando, en realidad no se esta interrumpiendo nada.
- Ambos tipos de interrupción trabajan con el mismo mecanismo, la **tabla de vectores**, que se usa para iniciarlas.

Ejecución de interrupciones

- La figura muestra la secuencia que ejecuta la maquina la ejecutar la interrupción 21h. Esta llama a las funciones del DOS, que son iniciadas de acuerdo al valor que contiene el registro AI.



Programando manejadores

- Para declarar en C una rutina de interrupción escribimos

```
void interrupt mi_manejador ()  
{  
    /* codigo*/  
}
```

- Para instalar el manejador en el lugar del vector 0x62

```
setvect(0x62, mi_manejador);
```

- Para declarar una variable en la cual una función pueda retornar un vector

```
void interrupt ( *vector_original) ();
```

- Para obtener y conservar el vector original y poderlo restaurar podemos escribir

```
vector_original = getvect ( 0x62 );
```

Tabla de vectores (Memoria)

- Existe un registro (idtr) que tiene la dirección base de la tabla y el largo
- Es posible definir hasta 256 entradas
- Se tiene 3 tipos de descriptores:
 - Task gate
 - Interrupt gate (interrupciones)
 - Trap gate (excepciones)
- La tabla es creada al iniciarse el sistema y luego los device drivers de los dispositivos la actualizan con los manejadores correspondientes
- Vector: dirección de cuatro bytes (CS:IP) de un manejador de interrupciones, que está ubicados en mem. convencional, ROM BIOS mem. superior, y COMMAND.COM.
- Los primeros vectores: interrupciones de hardware.
- Comienza 8 vectores mas allá que el nro de vector (int 0 empieza en 0000:0020h)
- 8 posiciones inferiores: tipo especial de interrupciones, las excepciones

Tabla de vectores (Memoria)

- Las ocho posiciones inferiores se reservan para un tipo especial de interrupciones, las excepciones.
- Son interrupciones iniciadas por el CPU cuando se producen errores que no puede manejar (división por cero, una instrucción que no reconoce, etc.).
- Hay interrupciones cuyos manejadores ya están programados en la BIOS, ubicados en la memoria alta en ROM como parte del hardware y las del DOS, en el COMMAND.COM.
- El procesador esta preparado para recibir interrupciones a través de su entrada INT o a través de la instrucción de igual nombre.
- Algunas interrupciones llevan a cabo varias tareas de acuerdo al valor de algún registro, por ejemplo el AH, indicara que función es llamada cuando se produce la INT1Ah.

Interrupciones

- Cuando una interrupción es detectada e informada al procesador, éste para la ejecución actual y ejecuta el código correspondiente de atención de interrupciones
- Esto genera un **kernel control path**, que ejecuta en la estructura del proceso que estaba ejecutando
- Los registros de ejecución son salvados en el Kernel Mode Stack del proceso
- La atención de interrupciones debe satisfacer:
 - Las interrupciones deben ser atendidas de forma rápida
 - Toda tarea que no sea necesaria se debe diferir
 - Poder ejecutarlas en forma anidada
 - Deben ejecutar, todo lo que sea posible, con las interrupciones habilitadas

Excepciones

- Las excepciones son generadas por:
 - Errores en la programación (p.ej.: división por cero)
 - Condiciones anómalas (p.ej.: fallo de página)
- Si son de errores de programación se le envía una señal al proceso que la generó, el proceso debe capturar esta señal, sino se ejecuta el manejador por defecto que finaliza ejecución
- Si se da una condición anómala, el núcleo deberá reestablecer la condición para continuar la ejecución
- Se clasifican en:
 - Detectadas por el procesador
 - Fallos: Pueden ser corregidos y retoman ejecución la instrucción que generó el fallo
 - Traps: utilizada, en mayor medida, para debugging
 - Aborts: un error grave ocurrió como un fallo del hardware

Excepciones

#	Exception	Exception handler	Signal
0	Divide error	<code>divide_error()</code>	<code>SIGFPE</code>
1	Debug	<code>debug()</code>	<code>SIGTRAP</code>
2	NMI	<code>nmi()</code>	None
3	Breakpoint	<code>int3()</code>	<code>SIGTRAP</code>
4	Overflow	<code>overflow()</code>	<code>SIGSEGV</code>
5	Bounds check	<code>bounds()</code>	<code>SIGSEGV</code>
6	Invalid opcode	<code>invalid_op()</code>	<code>SIGILL</code>
7	Device not available	<code>device_not_available()</code>	None
8	Double fault	<code>doublefault_fn()</code>	None
9	Coprocessor segment overrun	<code>coprocessor_segment_overrun()</code>	<code>SIGFPE</code>
10	Invalid TSS	<code>invalid_TSS()</code>	<code>SIGSEGV</code>
11	Segment not present	<code>segment_not_present()</code>	<code>SIGBUS</code>
12	Stack segment fault	<code>stack_segment()</code>	<code>SIGBUS</code>
13	General protection	<code>general_protection()</code>	<code>SIGSEGV</code>
14	Page Fault	<code>page_fault()</code>	<code>SIGSEGV</code>
15	Intel-reserved	None	None
16	Floating-point error	<code>coprocessor_error()</code>	<code>SIGFPE</code>
17	Alignment check	<code>alignment_check()</code>	<code>SIGBUS</code>
18	Machine check	<code>machine_check()</code>	None
19	SIMD floating point	<code>simd_coprocessor_error()</code>	<code>SIGFPE</code>

Manejadores de interrupciones

- El device driver de los dispositivos es el encargado de registrar la rutina de interrupción
- Esta tarea involucra invocar la rutina:

`request_irq`

`unsigned int irq`

`SA_INTERRUPT, SA_SAMPLE_RANDOM, SA_SHIRQ`

`irqreturn_t (*handler)(int, void *, struct pt_regs*)`

`unsigned long irqflags`

`const char * devname`

`void * dev_id`

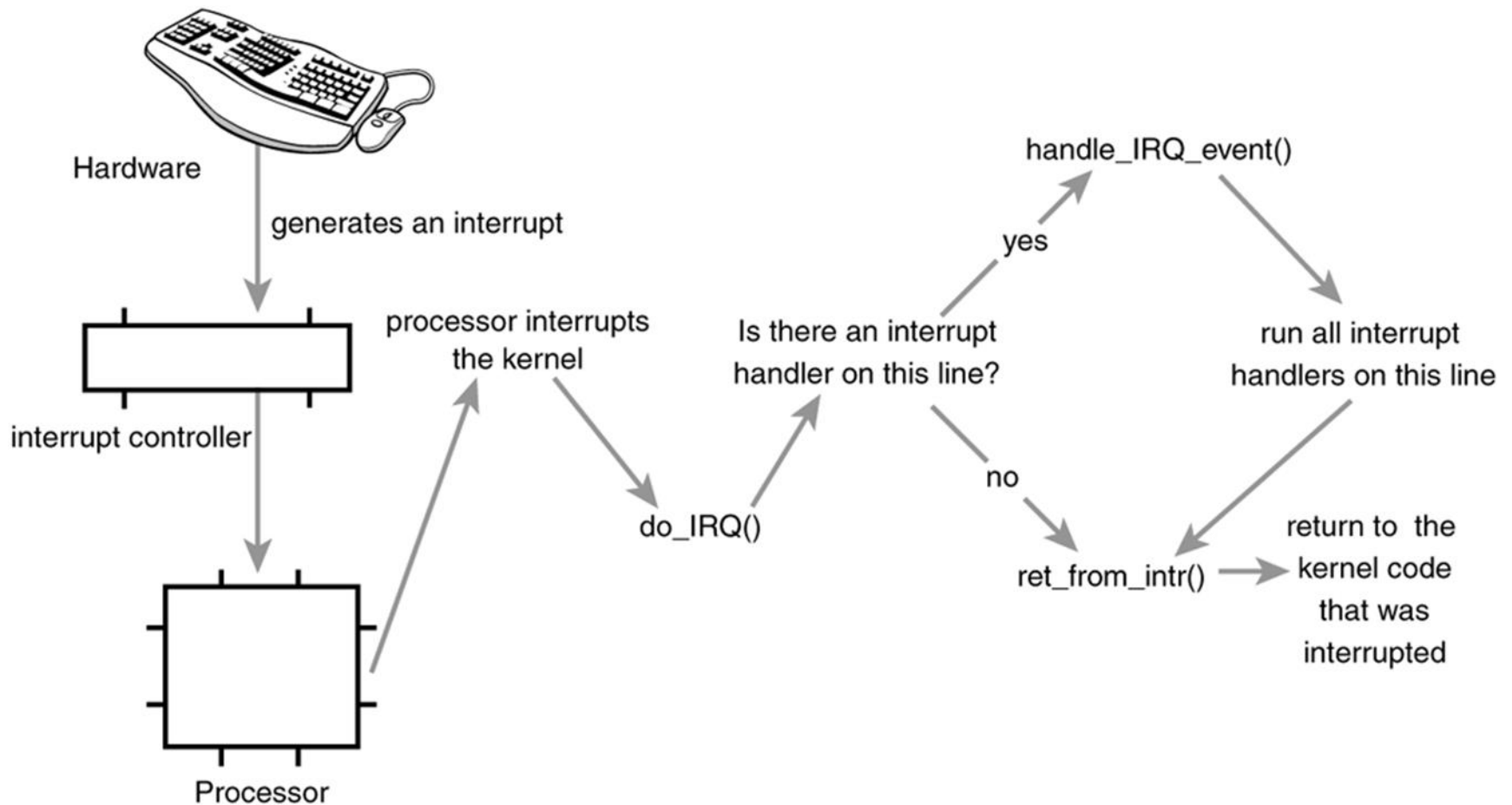
Donde:

- `irq`: Número de línea de interrupción
- `dev_id`: El que se le paso cuando se declaró el handler
- `regs`: estructura que contiene los registros antes de la interrupción

Manejadores de interrupciones

- El código debe ser bastante limitado
- Solo debería tomar en cuenta copiar la información a memoria y liberar el PIC
- No tiene necesidad de ser reentrante: la interrupción queda deshabilitada en procesador
- Es posible que en una línea se comparta por varios dispositivos. En estos casos el sistema invocará todas las rutinas correspondientes y solo deberá ejecutarse la del dispositivo adecuado
- La rutina hace uso del kernel mode stack del proceso interrumpido y, como se pueden anidar, se debe hacer un uso lo más limitado posible

Camino de una interrupción



Habilitación de interrupciones

- En el handler es posible deshabilitar las interrupciones en forma momentánea:

```
local_irq_disable() /* cli */
```

```
...
```

```
local_irq_enable() /* sti */
```

- También es posible deshabilitar alguna interrupción específica:

```
disable_irq(unsigned int irq)
```

```
enable_irq(unsigned int irq)
```